Contracts for C++: Postcondition Captures

Document #:	D3098R3
Date:	2026-10-29
Project:	Programming Language C++
Audience:	EWG, LEWG

Reply-to:	Timur Doumler <papers@timur.audio>
	Gašper Ažman <gasper.azman@gmail.com>
	Joshua Berne <jberne4@bloomberg.net>

Abstract

We propose the addition of captures to postcondition specifiers. With these *postcondition captures*, postcondition predicates can refer to values of parameters and other entities at the time when a function is called, and use those values when checking the postcondition predicate at the time when the function returns. Postcondition captures allow the user to assert common postconditions inexpressible in C++26, e.g., that `push_back` increments the size of the container by one. In addition, they enable using the value of a non-reference function parameter in a postcondition assertion without having to declare that parameter `const` on all declarations of the function and of all functions overriding it.

Contents

1 Revision history **3**

2 Motivation **3**

3 Overview **4**

4 History and context **6**
- 4.1 Referring to "old" values 6
- 4.2 Procedural interfaces 7
- 4.3 Non-reference parameters 7
- 4.4 Closure-based syntax 8

5 Discussion **8**
- 5.1 Contract assertions vs. lambda expressions 8
- 5.2 Name lookup 9
- 5.3 Kinds of captures and kinds of contract assertions 10
  - 5.3.1 Init-captures on postcondition assertions 10
  - 5.3.2 Explicit capture-by-copy on postcondition assertions 10
  - 5.3.3 Default capture-by-copy on postcondition assertions 11
  - 5.3.4 Capture-by-reference on postcondition assertions 11
  - 5.3.5 Capture this and `*this` on postcondition assertions 12

5.3.6 Captures on precondition assertions and assertion statements 13
5.4 Other static properties 13
5.4.1 const-ification 13
5.4.2 Capturing parameter packs 14
5.4.3 Ordering with respect to attributes 14
5.5 Evaluation model 15
5.5.1 Initialisation of captures relative to precondition assertions 15
5.5.2 Initialisation and destruction relative to other captures 16
5.5.3 Side effects, elision, and duplication 18
5.5.4 Destructive side effects in captures 19
5.5.5 Interaction between postcondition captures and the function body 20
5.6 Failure to evaluate a postcondition capture 21
5.6.1 Constructing or destroying the capture fails at runtime 21
5.6.2 Constructing or destroying the capture fails at compile time 22

6 Implementation Experience 23

7 Proposed wording 23

1 Revision history

R3 (July 2026 mailing):

- Completed draft wording
- Made clear that capture destruction can interleave with potcondition evaluations.
- Added discussion of alternatives for making captures `const` in predicates
- Added implementation experience
- Redeclaration matching for captures (section pending)
- Added a better, less scary code example for non-parameter init-captures
- Various minor fixes

R2 (May 2026 mailing, as presented to EWG on 2026-06-10):

- Removed capture-by-reference
- Changed capture initialisation and destruction ordering following SG21 guidance
- Rebased paper onto C++26 DIS
- Rewritten and expanded up-front overview section
- Various minor fixes, additions, and editorial changes

R1 (December 2024 mailing, as presented to SG21 on 2024-12-05):

- Rebased paper onto [P2900R11]
- Added discussion of name lookup, side effects, elision and duplication, interaction between postcondition captures and the function body, lifetime extension of temporaries, and failure to evaluate a postcondition capture
- Clarified `const`-ification rules in the initialiser of the postcondition capture
- Rearranged subsections to improve the logical flow of the paper
- Various other editorial changes

R0 (October 2024 mailing):

- Original version

2 Motivation

C++26 postcondition assertions, which are checked when a function returns, provide no way to refer to the state of the program at the time the function was called, such as the "old" value of a function parameter or the "old" state of the object of a member function. Yet, such postconditions are relatively common in C++ libraries including the C++ standard library — for example, the postcondition of `push_back` that, if the function returns normally, the size of the container is incremented by one. Such postconditions are inexpressible in C++26. Further, C++26 places

restrictions on using non-reference parameters in postcondition assertions. Such parameters must be declared `const` on all declarations of the function and any overriding functions, and the function (or any overriding functions) cannot be a coroutine:

```
int f(int i)
  post (r: r >= i); // error in C++26: i is not const
```

These restrictions are necessary to allow human readers, compilers, and static analysis tools to reason that a function will not modify the value of a non-reference parameter odr-used in a postcondition before that postcondition is checked, without having to inspect the function body (which might be arbitrarily complicated and/or inaccessible due to being in a different translation unit). Without these restrictions, postcondition assertions could spuriously pass where they should fail, or vice versa, with no apparent reason (for examples, see [P2900R14] Section 3.4.4, and [P3484R2] Section 1). However, requiring `const` on non-reference parameters as above is awkward and unintuitive, as users have long been taught that such `const` qualifiers on non-defining declarations have no observable effect. It also makes it impossible to use the value of a parameter in a postcondition assertion if, for whatever reason, that `const` cannot be added in all places required (for example, if there are overriding functions whose declaration and/or definition cannot be modified), or if the function must be implemented as a coroutine). This paper is proposing an extension to C++26 contract assertions that addresses all of the above limitations.

3 Overview

We propose to add a new, optional syntactic construct to postcondition assertions, called *postcondition captures*. Postcondition captures are spelled in the same fashion as lambda captures. They are placed immediately after the `post` contextual keyword and before the predicate:

```
post [captures] (predicate)
```

Postcondition captures are variables constructed when the function is called. They are visible only to the predicate of the postcondition assertion they are associated with. With a postcondition capture, postconditions that refer to the state of the program at the time the function was called — such as the postcondition of `push_back` that the size of the container is incremented by one — can be expressed as follows:

```
void push_back(const T& item)
  post [old_size = size()] (size() == old_size + 1);
```

The passed-in value of a non-reference function parameter can be used in a postcondition assertion without declaring the parameter `const`, by capturing that parameter instead (which results in a copy):

```
int f(int i)
  post (r: r >= i);          // error: i is not const
int f(int i)
  post [i] (r: r >= i);      // OK, capture i by copy
```

Further, postcondition captures on an overridden function remove the need to add `const` to any overriding functions and thus avoid remote code breakage. Finally, postcondition captures make it

possible to odr-use non-reference parameters in the postcondition assertions of a coroutine (which is ill-formed in C++26 because coroutine parameters will be moved-from on construction of the coroutine frame regardless of whether they are declared `const`):

```
generator<int> sequence(int from, int to)
  pre (from <= to)
  post [from, to] (g : g.size() == to - from + 1);
  // OK, use copies of the parameters made when function is called
```

However, we do not allow capture-by-reference. This would introduce several specification and implementation challenges (see Section 4.3.4) while not being all that useful in practice:

```
int f(int i)
  post [&i] (r: r >= i);       // error
```

We also do not allow default captures (see Section 4.3.3), or capturing `this` or `this` (see Section 4.3.5), as these are less useful on postcondition assertions than they are on lambdas. We also do not allow captures on `pre` or `contract_assert` (Section 4.3.6). Since postcondition captures are variables declared inside a contract assertion, not outside of it, they are not subject to `const`-ification, following the existing C++26 rule (see Section 4.4.1):

```
void increment (Iterator& iter)
  post [iter_old = iter] (++iter_old == iter); // OK, no const_cast needed
```

The proposed evaluation model (see Section 4.5) for postcondition captures is as follows. Evaluating a postcondition assertion with a postcondition capture involves three distinct phases:

- Constructing the postcondition captures;
- Evaluating the predicate of the postcondition;
- Destroying the postcondition captures.

While these phases happen at different points in time, they are part of the same postcondition-assertion evaluation, and thus should be thought of as a *single unit*: .

- All three phases of a given postcondition-assertion evaluation must be evaluated with the same evaluation semantic (`ignore`, `observe`, `enforce`, or `quick-enforce`).
- All three phases of a given postcondition-assertion evaluation must be performed either caller-side or callee-side (or not at all).
- If the postcondition-assertion evaluation is evaluated multiple times, each phase will be evaluated multiple times, and the side effects of each phase will be observed that amount of times.
- If the result of the predicate can be predicated without evaluating any of it, and thus the check can be elided, all three phases need to be elided.

Further, our proposed model considers all postcondition captures related to a particular sequence of function-contract assertions as local variables in a scope that contains that sequence (however postcondition predicates do not see the captures of other postconditions in the same sequence). All precondition assertions are evaluated and captures are initialised as the scope begins, then the

function is invoked. After the function returns, all postcondition assertions are evaluated, and finally the local variables (the captures) are destroyed when the scope exits. When precondition assertions and postcondition assertions are interleaved, these evaluations happen in lexical order (see Section 4.5.1):

```
int f(int i, int j)
  post [i] (r: r != i)
  pre (i > 0);
```

In this example, the postcondition capture is constructed `before` the precondition assertion is evaluated because it lexically appears before it. After any batch of postcondition assertions (such as the callee-side or caller-side postconditions) have been evaluated, destruction of the postcondition captures happens in reverse lexical order of declaration (see Section 4.5.2). It follows that, if any postcondition-predicate evaluations trigger calls to the contract-violation handler, this happens before any destruction of postcondition captures. If constructing or destroying the capture itself exits via an exception, the contract-violation handler is called; calling `detection_mode()` on the passed-in contract-violation object will return `detection_mode::evaluation_exception` as usual, and calling `kind()` will return `assertion_kind::post_capture`, a new proposed enum value (see Section 4.6.1). If a postcondition capture would copy or destroy an object that is not copyable or destructible, the program is ill-formed; if the postcondition capture is evaluated during constant evaluation but constructing or destroying it is not a core constant expression, a compile-time contract violation occurs as usual (see Section 4.6.2).

4 History and context

The need to refer to the state of the program at the time the function was called when checking the postcondition of that function arises frequently. It is such a common requirement that many different C++ Contracts proposals, starting with the very first one [N1613] over two decades ago, attempted to tackle this problem in various ways.

4.1 Referring to "old" values

One possible approach is to define an operator which, when applied to a named entity, yields the value that this entity had at the time the function was called. Such an operator was proposed many times: as a magic function `std::old` in [N1669], as a keyword `__old` in [N1773], as a keyword `oldof` in [N1866], and as a keyword `pre` (standing for "previous", not for "precondition") in [N4110]. If size is a data member of the container class, the postcondition of `push_back` that we have been using as an example can be expressed with such an operator as follows:

```
void push_back(const T& item)
  post (size == oldof(size) + 1);
```

However, if there is no parameter or variable called `size`, and the value has to be obtained via a function call `size()` instead, the operator approach does not provide a way to write this postcondition. It is therefore not a generic solution – a different approach is needed.

4.2 Procedural interfaces

When precondition and postcondition assertions are written as a block containing a sequence of statements, "old" values can be stored in a variable that is declared by the user inside that block, initialised before the function is called, and checked afterwards. In the hypothetical syntax for procedural interfaces [P0465R0] as shown in [P2961R2] Section 6.7, the aforementioned postcondition of `push_back` could be expressed as follows:

```
void push_back(const T& item)
interface {
  auto old_size = size();
  implementation;
  assert (size == old_size + 1);
}
```

This is the approach taken in [N1613], except that it was following the design of Contracts in the D programming language and thus the proposed syntax was slightly different from the above example. However, C++26 follows a different design, where each precondition and postcondition is specified separately by a `predicate` – a single expression that evaluates to `true` if the contract is satisfied. In this design, there is no place where one could declare the variable `old_size`, so this approach does not work for C++26 contract assertions. We might add procedural interfaces as an extension to C++ some time in the future, but this is not currently planned in the C++29 timeframe (see [P3850R0]). The ability to refer to the state of the program at the time the function was called when checking its postcondition assertions is too important to be delayed until procedural interfaces are adopted or to have as verbose an interface for common patterns.

4.3 Non-reference parameters

The problem of breaking postcondition assertions by modifying parameter values in the function body was also already recognised in [N1613]. In that proposal, a parameter was `implicitly const` if it was odr-used in a postcondition. In C++2a Contracts [P0542R5], modifying a parameter odr-used in a postcondition was instead specified to be undefined behaviour. From today's perspective, neither of these options are viable. Instead, C++26 requires the parameter to be explicitly declared `const` on `all` declarations of the function, as well as all declarations of *all overrides* of the function (for an explanation why it is problematic to require `const` on the definition only or make the `const` implicit, see discussion of Option V4 in [P3484R2]). Further, a function that odr-uses a non-reference parameter in a postcondition assertion cannot be a coroutine, because coroutines modify their parameters even if they are declared `const` by the user (see [P2957R2] and [P3387R0]). Unfortunately, all of these restrictions have significant tradeoffs. Having to declare a non-reference parameter `const` on all declarations is an unusual restriction: in today's C++, the `const` has no meaning on a non-defining declaration and is almost never written. Further, extending the `const` requirement to all declarations of all overrides can lead to remote code breakage: adding a postcondition assertion to a virtual function will break any overrides of that function and any code using those overrides (see discussion of Option V2 in [P3484R2]). Finally, adding such a postcondition assertion to a coroutine requires making the function a non-coroutine, for example by wrapping the original coroutine implementation into a non-coroutine wrapper (see example in [P3387R0] Section 2), and making an explicit parameter copy in that wrapper. There are cases where none of the above limitations and workarounds are acceptable. However, in C++26, the only alternatives to the above are to make the

parameter a reference parameter – which will often make the API worse and prevent optimisations, discouraging the introduction of contract assertions – or to not add the postcondition assertion at all.

4.4 Closure-based syntax

A generic and complete solution to this problem was first proposed in the closure-based syntax proposal, [P2461R1]. That paper was primarily a proposal for a basic contract-assertion syntax, meant to replace the attribute-like syntax (`[[pre: x]]` and `[[post: x]]`) that was the main proposal at the time. The ability to add a lambda-like capture to a contract assertion was included in [P2461R1] as a follow-up extension. The closure-based syntax proposal had a design issue: it placed the contract predicate inside braces `{...}`, even though C++ usually surrounds expressions with parentheses `(...)` and statements with braces `{...}`, and the predicate is an expression. This made a predicate look like the body of a lambda expression — an entirely different kind of construct. Therefore, after long deliberation, the "natural syntax" [P2961R2], which places the contract predicate inside parentheses, was adopted instead. However, the idea of postcondition captures from [P2461R1] was recognised as a very desirable feature (see [P2885R3]). Therefore, C++26 contract assertions have been explicitly designed to seamlessly accommodate this extension, and we now propose to include it in C++29.

5 Discussion

5.1 Contract assertions vs. lambda expressions

Postcondition captures are conceptually similar to lambda captures, which allows us to leverage a familiar syntax. However, contract assertions and lambda expressions themselves are distinct C++ features with a number of important differences. These differences exist because lambda expressions are objects that have their own lifetime and are designed to be passed around, while contract assertions are not — they are correctness checks attached to a function declaration. While lambda captures are effectively data members of that object, postcondition captures should instead be thought of as local variables introduced into the scope of the contract assertion. Due to these differences, we would be ill-advised to simply apply the semantics of lambda captures "as is" to contract assertions. The first and most obvious difference is that the body of a lambda is a block that can contain an arbitrary sequence of statements, and the value of evaluating the lambda is determined by its `return` statement. On the other hand, the "body" of a contract assertion, its predicate, is not a statement but a single expression that is contextually converted to `bool`. The second difference is that the definition of a lambda does not say anything about if and when the lambda will be called; it might happen at any later time, in any other context, and on any other thread. On the other hand, it is defined very precisely when a contract assertion is evaluated: when the function is called (`pre`), when control flow reaches the contract assertion (`contract_assert`), or when the function returns control to the call site (`post`). Therefore, if we were to consider contract assertions with captures as a special kind of lambda, `pre` and `contract_assert` would be akin to always-immediately-invoked lambdas, whereas `post` would always be like a lambda invoked at the end of the block in which it is declared. Finally, the third difference is related to the second: inside the body of a lambda, variables with automatic storage duration from the enclosing scope cannot be used directly (only captured), nor would it make sense to do so, as they would possibly no longer

be alive by the time the lambda is called:

```
int a = 0;       // global
class X {
  int b = 1;     // member
  void test() {
    int c = 2;   // uncaptured local
    int d = 3;   // captured local
    auto f = [d] {
      /*  you can refer to a here, but not to b or c;
      referring to d refers to the captured variable,
      not the one in the outer scope */ };
  }
};
```

On the other hand, for a contract assertion, such lifetime issues cannot arise, not even for postcondition assertions – all variables that are alive when the function is called are still alive when the postcondition is checked. Therefore, there is no reason to "hide" variables from a contract assertion predicate, and name lookup in the predicate sees the same entities as the function body does:

```
int a = 0;
class X {
  int b = 1;
  void test()
    pre (/* you can refer to a and b here */)
    post (/* you can refer to a and b here */)
  {
    int c = 2;
    int d = 3;
    contract_assert (/* you can refer to a, b, c, and d here */);
  }
};
```

5.2 Name lookup

Due to the fundamental differences between lambdas and contract assertions described in Section 4.1, the interaction between postcondition captures and name lookup in the predicate will necessarily have to work differently from the interaction between lambda captures and name lookup in the function body. In particular, postcondition captures introduce new variables into the scope of the predicate that can now shadow variables from outer scopes that would otherwise be found in the predicate by name lookup. Consider:

```
bool b = true;
void f1() post (b) {        // (b) refers to outer b, evaluates to false
  b = false;                  // modifies outer b
}
void f2() post[b = b] (b) { // (b) refers to capture, evaluates to true
  b = false;                // modifies outer b
}
```

On the other hand, within the postcondition capture itself, variables are not shadowed by other captures:

```
int i = 1;
int j = 2;
void f() post [i = 3, j = i] (j == 1) {} // j == 1 evaluates to true
```

This is identical to how captures work for lambdas:

```
int i = 1;
int j = 2;
int k = [i = 3, j = i] {
  return j;  // returns 1, not 3
}();
```

5.3 Kinds of captures and kinds of contract assertions

For our proposal, we need to decide which subset of lambda captures (init-captures, explicit and implicit captures by copy, explicit and implicit captures by reference, capturing `this`) makes sense for which kinds of contract assertions (`pre`, `post`, `contract_assert`). We explore this question in this section.

5.3.1 Init-captures on postcondition assertions

The ability to write init-captures on postcondition assertions is the "must-have" minimal feature to provide the functionality missing in C++26. These init-captures are allowed to capture arbitrary entities, not just function parameters, which enables a large range of use cases. For example, we can record the input size of a container rather than having to copy the whole container. We can store the underlying pointer value of a smart pointer, rather than having to copy the whole pointer (which may not be possible, e.g. `std::unique_ptr`). We can also record any other program state at the time of the function call, such as current values of shared variables, the current time, etc.:

```
// function must wait for at least as long as specified
void wait(time wait_time)
  post [start_time=gettime()] (gettime() - start_time >= wait_time);
```

Many other motivating use cases for such init-captures on postcondition assertions are given in [P2461R1].

5.3.2 Explicit capture-by-copy on postcondition assertions

The ability to capture explicitly by copy is not strictly necessary, because the same effect can be achieved with an init capture. However, it is a familiar shorthand syntax and users will expect it to work:

```
// Capture-by-copy syntax
int min(int x, int y)
 post [x, y] (r: r <= x && r <= y);
// Equivalent init-capture syntax
int min(int x, int y)
 post [x=x, y=y] (r: r <= x && r <= y);
```

We do not see any benefit in making the syntax on the left-hand side ill-formed, as that would simply impose more typing on the user to achieve the same result. We therefore propose that it should be allowed. The next question is whether we want to allow capturing variables other than function parameters by copy with the above syntax. However, every capture-by-copy introduces a name to the predicate context that shadows another name which otherwise `would` be available in the postcondition predicate, and unlike for lambdas, this shadowing also happens for `local` variables. This makes it more difficult to reason about the meaning of the predicate. For parameters, this is not too much of a concern, because the name that is being shadowed is right there both in the function parameter list and in the capture, and the object being shadowed will go out of scope anyway after the postcondition predicate is evaluated and the function returns. However, all other variables are defined farther away. Shadowing them by capture would therefore make it more difficult to reason about what name is being shadowed in the predicate, what it means, and which object is being accessed where. Therefore, we propose that only function parameters should be captured-by-copy in a postcondition capture:

```
namespace X {
  int i = 0;
  int f1()      post [i] (r: r > i); // error: cannot capture non-param i
  int f2(int j) post [j] (r: r > j); // OK
};
```

5.3.3 Default capture-by-copy on postcondition assertions

We propose that default captures on contract assertions should be ill-formed:

```
int min(int x, int y)
  post [=] (r: r <= x && r <= y);  // error
```

The benefit-cost ratio for allowing this variation, even for function parameters, does not seem quite as favourable as for explicit capture-by-copy. Yes, we get an even shorter syntax for operating on copies of non-`const` non-reference parameters in the postcondition predicate; however, this syntax makes it more difficult to read and interpret the predicate correctly. If the capture is explicit (either as an init-capture or as an explicit by-copy capture), the name of every copied object appears in the capture immediately left of the predicate, which is easy to see. However, if the capture is implicit, it is no longer immediately obvious which variables appearing in the predicate refer to the original object and which are copies local to that predicate. Remember that for lambdas, this ambiguity can never arise: if the variables in question were not captured, they would not be accessible in the body of the lambda at all.

5.3.4 Capture-by-reference on postcondition assertions

We propose to make all forms of capture-by-reference in contract assertions (whether explicit or implicit) ill-formed:

```
void f(int i)
  post [&i] (r: r > i);    // error
  post [&i=i] (r: r > i);  // error
```

If we allowed capture-by-reference of parameters, we would have to guarantee that the caller side and the callee side always refer to the same parameter object. This does not work well with parameter types that can be passed via registers (see [P3487R0]): despite holding a reference to the parameter object, the postcondition assertion is not going to consistently observe changes that the function body does to its value, as it might be operating on a temporary copy ([class.temporary] paragraph 3). Further, if the function is implemented as a coroutine, a reference to a parameter in a postcondition assertion would refer to a moved-from object ([dcl.fct.def.coroutine] paragraph 14), which is arguably surprising and hostile to the user. Finally, we would be left with two choices — both suboptimal — regarding the relationship of these references to the lifetime of the parameter objects. The first would be to require that the referenced object's lifetime always extend past the end of the evaluation of the postcondition assertion. While standard C++ does guarantee that today (postcondition assertions are evaluated before parameter destruction, see [expr.call]), that guarantee does not always hold with compiler extensions such as types with the `[[clang::trivial_abi]]` attribute (e.g., libc++'s `std::unique_ptr`) which may be relocated (ending the lifetime of the original parameter object). This restriction would then hinder adoption of such advancements on top of potential future trivial relocation proposals — but only when postcondition assertions with such by-reference captures are present. This would lead to an ABI change resulting from the presence of a contract assertion, a clear violation of the Contracts Prime Directive ([P2900R14] Section 3.1.1). The other alternative is to introduce no such restrictions, which would lead to dangling references in the same situations — a new potential footgun in the language that does not seem to carry its weight. For non-parameters, the above issues do not arise, but their utility is questionable. Capture-by-reference of an `id-expression` would essentially do nothing: due to the name lookup rules for contract predicates (see Section 4.2), an `id-expression` referring to an object other than the parameters that would be accessible in the postcondition capture for capturing it by reference is `already` accessible in the postcondition predicate directly without having to capture it. The only effect that the capture-by-reference would have is that for any identifier `x` of non-reference type captured in this way, `decltype(x)` would change from `T` to `T&`. This does not seem useful. Capture-by-reference of a temporary or rvalue reference is equally problematic, as lifetime extension rules are already complicated enough and might not apply intuitively. Capturing an lvalue reference returned by an expression — such as the results of a lookup into a container— might be useful if that lookup would produce a different result after the function executes, but the same semantics can be achieved by capturing a pointer to that object instead of a reference:

```
void f(int key, std::map<int, int>& m)
   pre (m.find(key) != m.end())
   post [&ref=*m.find(key)] (&*m.find(key) == &ref) // error: ref capture
   post [ptr=&*m.find(key)] (&*m.find(key) == ptr); // OK, address stable
```

This would not limit functionality while making it more clear that the address being captured is a relevant component of what is being checked and when it is being evaluated.

5.3.5 Capture this and *this on postcondition assertions

In most cases, capturing `this` in a postcondition assertion assertions would have no effect whatsoever: due to the name lookup rules for contract predicates (see Section 4.2), any member or member function that is accessible in the body of the function in question is `already` accessible in the postcondition predicate as well. The only exception to that is the naming of member variables

in the postcondition assertion of a destructor, where using the value of such a variable will lead to undefined behaviour (see [P3510R1]). We certainly do not want to make such use easier via postcondition captures. We therefore propose to make it ill-formed to capture `this` in contract assertions. Note that this is again a consequence of the difference between lambdas and contract predicates discussed in Section 4.1. Capturing `this` by copy does not make sense either because that would make it hard to reason about whether members named in the postcondition predicate refer to the captured or the current version of `this`. Therefore, we propose to make capturing `this` ill-formed as well. Again, the desired effect can be achieved with an init-capture (Section 4.3.1):

```
class DeckOfCards {
  // ...
  void shuffle()
    post [old = *this] (std::ranges::is_permutation(old, *this));
};
```

5.3.6 Captures on precondition assertions and assertion statements

So far we have only considered postcondition assertions. It has been suggested that captures could be added to precondition assertions and assertion statements as well. However, for these other kinds of contract assertions, captures would necessarily behave rather differently. At first, captures on `pre` and `contract_assert` seem useful for generating local copies that can be accessed only in the predicate, for example if the predicate check requires modification of a parameter value but we do not want to modify the original object:

```
void (Iter begin, Iter end)
  pre [begin, end] (begin != end && ++begin != end);
```

However, for `pre` and `contract_assert`, the predicates of these contract assertions are checked immediately after such a capture is constructed; there is no temporal separation between the two as there is for `post`. Therefore, captures on `pre` and `contract_assert` would not actually provide any new functionality; they would just be a shorthand syntax for something we can already write in C++26 using immediately-invoked lambda expressions:

```
void (Iter begin, Iter end)
  pre ([begin, end] { return begin != end && ++begin != end; }());
```

We conclude that captures for `pre` and `contract_assert` are non-essential syntactic sugar that would complicate specification and implementation. In addition, the difference in behaviour between captures for `post` (which "saves" values for later, something that can otherwise not be done) and captures for `pre` and `contract_assert` (which do no such thing) would likely be not immediately obvious to users. We therefore propose to make captures on any contract assertions other than `post` ill-formed.

5.4 Other static properties

5.4.1 const-ification

To discourage destructive side effects in contract predicates, C++26 makes `id-expression` s that refer to objects declared outside of the contract assertion `const` by default. However, this `const`-ification

does not apply to objects declared inside the contract assertion, as they are not observable outside of the cone of evaluation of the contract assertion and thus highly unlikely to cause destructive side effects. Postcondition captures declare objects that are only observable within the predicate of the postcondition. Following the logic above, we therefore do `not` apply `const` by default for the captured entities, following the original proposal in [P2461R1]. For example, the following code is well-formed, without requiring a `const_cast` around `iter_old` in the predicate:

```
void increment (Iterator& iter)
  post [iter_old = iter] (++iter_old == iter);  // OK
```

The above semantics seem at odds with lambda captures, which do have `const` applied by default unless the lambda is declared mutable. However, as discussed above, a contract predicate is rather different from a lambda. First, it is just a single expression converted to `bool` and therefore much simpler than a lambda body, so any possible mutations will be more obvious. Further, if a developer decides to explicitly capture an object for local use inside the predicate, it is very likely that any mutation of this local object will be intentional. Finally, part of why a lambda's call operator is `const` is to make it not relevant whether it is evaluated on a lambda or a separate copy of a lambda: when storing callbacks, this can be the source of subtle bugs and surprises when copies of the same function are stored in different places. Postcondition assertions will never suffer from that: each set of postcondition captures is initialised exactly once and then used at most once, in pairs together. Therefore, `const` would not prevent any of the lambda-specific concerns that lead to lambda call operators being `const`, nor are they relevant for postcondition assertions. While the postcondition captures themselves are not `const`-ified, objects declared outside of the contract assertion are still `const`-ified as usual even if they appear in the initialiser of the postcondition capture:

```
int i = 0;
void f()
  post [j = ++i] (j);  // error: cannot modify const lvalue i
```

5.4.2 Capturing parameter packs

The grammar for lambda captures allows capturing parameter packs, both as init-captures and as explicit capture-by-copy. We see no good reason to disallow this for postcondition captures, with the same grammar. This avoids unnecessary inconsistency and can be occasionally useful:

```
template <typename... Args>
void test(Args... args)
  post [args...] (r: ((r < args) && ...));  // capture-by-copy
template <typename... Args>
void test(Args... args)
  post [...x=args] (r: ((r < x) && ...));   // init-capture
```

5.4.3 Ordering with respect to attributes

Unlike with lambda captures, postcondition captures appear syntactically `after` attributes appertaining to the contract assertion:

```
int f(int i)
  post [[vendor::always_enforce]] [i] (r: r >= i);
```

The rationale is that the capture relates to the predicate, and affects the meaning of the predicate expression, so it should be as close as possible to it, while the attribute relates to the entire contract assertion, and may affect properties unrelated to the predicate expression such as the evaluation semantic or the error message (see [P3088R1]) so it should be as close as possible to the introducing keyword (`pre`, `post`, or `contract_assert`). These syntactic rules follow the direction set by the C++26 contract assertion syntax [P2961R2], which was designed from the start to accommodate postcondition captures.

5.5 Evaluation model

While evaluating a captureless C++26 postcondition assertion only involves evaluating its predicate (or not), evaluating a postcondition assertion with a postcondition capture now involves three distinct phases:

- Constructing the postcondition captures;
- Evaluating the predicate;
- Destroying the postcondition captures.

In this section, we discuss how these phases should be sequenced relative to each other and relative to the evaluation of other precondition and postcondition assertions and other evaluations in the program. The proposed evaluation model follows two key design principles. The first design principle is that, even though evaluating a postcondition assertion now involves these three distinct phases, the postcondition assertion should still be thought of as a *single unit* with regards to properties such as: what evaluation semantic it is evaluated with (`ignore`, `observe`, `enforce`, or `quick-enforce`), whether it is checked caller-side or callee-side, whether it is elided or duplicated and any side effects are observed, etc. The second design principle is that all postcondition captures related to a particular sequence of function-contract assertions are considered to be akin to *local variables* in a scope that contains that sequence. As the scope begins, all precondition assertions are evaluated and postcondition captures are initialised, then the function is invoked. The function may then move on to another scope containing another sequence of function-contract assertions, for example, when passing from caller-side checks to callee-side checks. After the function returns, all postcondition assertions are evaluated, and finally the local variables (the captures) are destroyed when the scope exits. These two principles make the proposed feature easy to teach and understand. Its exact semantics, described in the remainder of this paper, all fall out from them.

5.5.1 Initialisation of captures relative to precondition assertions

When a function is called, the precondition assertions are evaluated and the postcondition captures are constructed. We need to specify the relative order of the two. In particular, C++26 allows interleaving precondition and postcondition assertions, i.e. having postcondition assertions that lexically precede precondition assertions:. This allowance exists because precondition and postcondition assertions often come in semantically-related pairs or groups and the user might want to group all assertions that belong to such a group together (possibly even wrapped by a macro). The question then arises what the order of evaluation should be if a postcondition assertion with a capture lexically precedes a postcondition assertion? Consider:

```
int f(int i)
```

```
  post [i] (r: r != i)
  pre (i > 0);
```

Should the postcondition capture be constructed before or after the precondition is checked? On the one hand, it seems that checking all precondition assertions first, and only then constructing the postcondition captures, is the more defensive choice as it avoids committing to constructing any postcondition captures prior to checking all of the precondition assertions of the function. This also matches the original proposal in [P2461R1], which says that no postcondition capture is constructed before all precondition assertions are checked. On the one hand, following the lexical ordering gives a clearer intuition for when evaluations will happen, including initialising captures, and so we follow that lexical ordering here. The initialisation of a postcondition-assertion capture happens as part of evaluating the precondition assertions of the function, in the same lexical ordering that the postcondition assertion is in relation to the precondition assertions of that function. This maintains the concept that precondition assertions (when enforced) will guard against evaluating code that follows them while not introducing surprising reorderings on top of that. Note that this order also impacts how captures will be initialised in virtual function calls. In a virtual function call — assuming the adoption of `pre` and `post` on virtual functions as proposed in [P3097R2] — the caller-facing and callee-facing contract-assertion sequences are still evaluated separately with captures initialised as part of one group or the other. Consider:

```
struct X {
  virtual int f(int i)
    post [i] (r: r != i)  // (1)
    pre (i > 0);          // (2)
};
struct Y : X {
  int f(int i) override
    post [i] (r: r > i)   // (3)
    pre (i % 2);          // (4)
};
```

In the above example, the evaluation order is as follows:

- The `caller`-facing postcondition capture at (1) is constructed,
- The `caller`-facing precondition at (2) is evaluated,
- The `callee`-facing postcondition capture at (3) is constructed.
- The `callee`-facing precondition at (4) is evaluated.

5.5.2 Initialisation and destruction relative to other captures

In which order should postcondition captures be constructed and destroyed relative to each other and to other evaluations? Consider:

```
void f()
  post [a = get_a(), b = get_b()] (a == b)   // (1)
  post [c = get_c(), d = get_d()] (c == d);  // (2)
```

From a language design perspective, the following four properties seem desirable:

1. Construct the captures in their lexical order: `a`, `b`, `c`, `d`;
2. Check the postcondition predicates in their lexical order: (1), then (2);
3. Destroy the captures in the reverse order of construction;
4. Destroy the captures associated with each postcondition predicate immediately after evaluating that predicate.

However, it turns out to be logically impossible to satisfy all four properties simultaneously. We can only satisfy at most three of them, but must give up one. Giving up property A seems highly unintuitive and surprising to users. Clearly, within a `single` *capture list* the initialisation order must match the lexical order — `a` must be constructed before `b`, and `c` must be constructed before `d`. Relaxing this requirement across `different` postcondition assertions — initialising `c` and `d` in that order, but before `a` and `b`, in order to satisfy all of the properties B, C, and D — seems somewhat less bad, and was considered in a previous revision of this paper, but was explicitly rejected by SG21. Giving up property B — i.e., reversing the order in which postcondition predicates are checked — seems equally unintuitive and surprising. Consider:

```
int* g(int* p)
  pre (p)
  pre (*p > 0)
  post [p] (*p > 0)          // (1)
  post (r: r)                // (2)
  post [p] ((*r / *p) > 1); // (3)
```

In the above example, it is reasonable to expect that the postcondition predicate at (1) will be checked before the predicate at (3) since (3) depends on (1). Having to write these postcondition assertions in the opposite lexical order to achieve the desired effect would introduce a massive new footgun. Furthermore, it would require a breaking change to C++26. Therefore, this option is out of the question. Giving up property C would also be unfortunate because this would break fundamental assumptions about the lifetimes of C++ objects being nested, rather than overlapping, in any given scope. Since `a`, `b`, `c`, and `d` are effectively local variables with automatic storage duration, they should always be destroyed in the reverse order in which they were constructed, whatever that order happens to be. Thus, we propose to give up property D, and to evaluate the postcondition-assertion sequence of function `f` at the beginning of this section in the following order:

- Construct `a`, then `b`;
- Construct `c`, then `d`;
- Execute the body of `f`;
- Check the postcondition predicate at (1);
- Check the postcondition predicate at (2);
- Destroy `d`, then `c`;
- Destroy `b`, then `a`.

This evaluation order fits seamlessly into our proposed programming model, which considers the postcondition captures to be local variables in a scope that encompasses the entire sequence of

function-contract assertions that the given postcondition assertion belongs to, rather than a scope that encompasses only the given postcondition assertion itself.

5.5.3 Side effects, elision, and duplication

We do not propose any changes to the rules for side effects, elision, and duplication of contract assertion evaluations. However, construction of the captures, evaluation of the predicate, and destruction of the captures are all considered parts of the `same` postcondition assertion evaluation (and not independent evaluations, even though they are separated in time). The program must observe either all or no side effects resulting from the entire evaluation; it is not allowed to selectively elide or duplicate some parts of that evaluation but not others.[1]

Concretely, the program is not allowed to observe only the side effects of constructing and destructing the postcondition capture but not those of the associated postcondition predicate or vice versa. If the implementation decides to elide or duplicate the evaluation of a postcondition predicate, it must then also elide or duplicate the construction and destruction of the associated postcondition captures. Conversely, if the postcondition predicate is a provably pure expression that can be trivially elided, but the compiler cannot prove the same property for the constructor and destructor of the capture, and the evaluation semantic is not `ignore`, it must perform construction and destruction the capture. Consider:

```
int i = 0;
bool f() { ++i; return true; }
struct X {
  X() { f(); }
  ~X() { f(); }
};
void g() post [x = X{}] (f()) {}
int h() {
  g();
  return i;
}
```

In the above program, `h()` may return 0, 3, 6, 9 ... but not any integer indivisible by 3. Finally, we need to specify how constructing postcondition captures fits into the rules for sequences of evaluations and repeated evaluations. This is necessary to clearly specify the behaviour for all possible combinations of precondition and postcondition assertions being checked caller-side or callee-side, respectively.[2] Consider again the example from Section 4.5.1:

```
int f(int i)
  post [i] (r: r != i)  // (1)
  pre (i > 0);          // (2)
```

Now, let us consider a highly unusual configuration: the engineer configures the build in such a way that the precondition assertion at (2) is checked callee-side, but the postcondition assertion at

[1] Note that this rule already exists in C++26 today: the implementation is not allowed to selectively elide or duplicate some subexpressions of a contract-assertion predicate but not others.

[2] Note that the concept of caller-side and callee-side checks relates to where the actual checks are laid down by the compiler. This is entirely different from the concept of caller-facing and callee-facing contract assertions used in [P3097R2] to define the semantics of virtual function contracts.

(1) is checked caller-side. Since the postcondition capture at (1) is part of the same postcondition assertion as the predicate at (1), the capture must be constructed caller-side as well. At first, it seems that an order of evaluation where the capture at (1) is constructed `after` the check at (2) would be impossible to implement. However, upon deeper inspection, this situation is perfectly compatible with the C++26 rules and those proposed here. The key is to realise that constructing postcondition captures is *a part of the precondition assertion sequence* of a function call, and to remember that whenever the evaluation of a function contract assertion sequence is repeated, an evaluation with the `ignore` semantic also counts as an evaluation. Once we realise this, it becomes clear that when the program is configured as described above, what is actually happening is the following:

- The precondition assertion is evaluated caller-side with the `ignore` semantic,
- The postcondition capture is constructed caller-side with the `enforce` semantic,
- The precondition assertion is evaluated again callee-side with the `enforce` semantic,
- The postcondition capture is constructed again callee-side with the `ignore` semantic,

Thus, this case is just one possible variation of function contract assertion sequence duplication. In practice, the postcondition capture will end up being constructed `before` the precondition is checked, not after, since the precondition evaluation preceding the postcondition capture is an `ignored` one. This might seem surprising, but it is exactly what the user asked for when they configured their program to be compiled in this peculiar way.

5.5.4 Destructive side effects in captures

As described in [P2900R14] Section 3.1 "Design Principles", contract assertions are supposed to observe the state of the program, but not change it, particularly in a way that could influence the correctness of the program that the contract assertions are supposed to assert. Contract assertions that violate these principles are said to have *destructive side effects* and are not a correct use of the Contracts facility. Destructive side effects are not synonymous with *side effects* in the core language sense (modifying objects and performing I/O). A contract predicate can have destructive side effects even if it has no side effects in the core language sense; conversely, a contract predicate can have side effects in the core language sense, even those that are observable outside of the cone of its evaluation, and yet not have destructive side effects. Consider:

```
void f(const std::string& message)
   pre([]{
     std::ostringstream todiscard;
     return parse(message, &todiscard);
   }() == SUCCESS);
```

Evaluating the above contract predicate may cause a dynamic memory allocation, lock a mutex, interact with `errno`, etc., which is perfectly fine as long as the developer deems those side effects not destructive for their particular program. A postcondition capture will perform a copy, which creates a new opportunity to introduce destructive side effects to a program. Consider:

```
struct Widget {
  Widget() { /* ... */ }
```

```
  Widget(const Widget& other) { /* ... */ }  // (1)
  ~Widget() { /* ... */ }                    // (2)
  // ...
};
Widget f(Widget w)
  post [w] (ret: ret.id() == w.id());
```

In this program, it is up to the developer to ensure that the copy constructor (1) and destructor (2) of `Widget` will not have any destructive side effects when evaluated as part of the postcondition capture mechanism.

5.5.5 Interaction between postcondition captures and the function body

Even if the postcondition capture itself does not have any destructive side effects, it can interact with program state outside of the postcondition assertion, which in turn can lead to surprising behaviour. For example, the copy created in the postcondition capture can observe modifications of program state that happen in the *function body*, i.e., in between evaluation of the capture and evaluation of the associated predicate. In particular, this will happen when attempting to capture an object of a non-regular type that appears to provide value semantics but actually provides reference semantics. Consider a type `Box` that encapsulates a value stored on the heap:

```
template <typename T>
class Box {
  std::shared_ptr<T> _storage = std::make_shared<T>();
public:
  // Default/copy/move ctor, dtor, assignment all compiler-generated
  Box(T value) { *_storage = value; }
  // Getters/setters
  T getValue() const     { return *_storage;  }
  void setValue(T value) { *_storage = value; }
  operator T() const     { return getValue(); }
  // To make an actual (deep) copy of the object:
  Box makeCopy() {
    Box copy;
    copy.setValue(getValue());
    return copy;
  }
};
```

It is questionable whether such irregular data types represent good design, but they are quite common in practice. If such a type is captured-by-copy in a postcondition capture, and then the underlying data is modified in the function body, the postcondition predicate will observe the modified data, even though it operates on a copy of the original parameter object:

```
void append(Box<CharArray> str, Char c)
  post [oldStr = str] (str == oldStr + c);  // will spuriously fail
```

Such behaviour subverts the intent of postcondition captures, but we cannot really do anything about it as the regularity of a type is not a compile-time-checkable property. It gets even more tricky if the `Box` class defined above is used in a generic piece of code:

```
template <IntegerLike T>
int half(IntegerLike  i)
  pre(i >= 0)
  post(r : r >= 0)
  post [i] (r: r <= i) // Capture i because I want to modify it below
{
  int h = i / 2;
  i.setValue(0);       // Now I can modify i, right?
  return h;
}
int main() {
  Box<int> n(42);
  auto h = half(n);
}
```

In this example, the postcondition assertion will see the modified value of `i` and will therefore spuriously fail. Yet, the source of the problem is not immediately apparent, and it is not immediately clear who is responsible: the author of `Box`, the author of `half`, or the user who used one in conjunction with the other. This property of postcondition captures is therefore a caveat that needs to be taught explicitly: capturing a function parameter by copy will copy-construct the object that the postcondition predicate will observe and "you get what you get" from that copy-constructed object, including false positives and false negatives if you are using a type with irregular copy semantics. Note that `const`-ification of the capture, if we were to propose it (we do not — see Section 4.4.1), would not help in such cases.

5.6 Failure to evaluate a postcondition capture

In this section, we discuss the different possible failure modes of postcondition captures and how they are handled in our proposed model.

5.6.1 Constructing or destroying the capture fails at runtime

Constructing and destroying postcondition captures is part of evaluating the associated postcondition assertion. Therefore, if any such construction or destruction fails, this should be treated as a contract violation, and in particular a violation of `that` postcondition assertion (just like a failed predicate check is a violation of that postcondition assertion). If constructing or destroying a postcondition capture exits via an exception, and the evaluation semantic of that postcondition assertion is `observe` or `enforce`, the contract- violation handler is called, and the function `contract_violation::detection_mode()` returns the enumeration value `detection_mode::evaluation_exception` as usual. To help the user distinguish between violations originating from evaluating the capture and violations originating from evaluating the predicate, we propose that the function `contract_violation::kind()` should return a new enumeration value `assertion_kind:: post_capture` rather than `assertion_kind::post` in this case. If the violation occurred while constructing a postcondition capture, the contract-violation handler returns normally, and the evaluation semantic is `observe`, any already constructed postcondition captures for `that` postcondition assertion are destroyed before execution continues, and further evaluation of that postcondition assertion is abandoned, i.e., its predicate will not be checked on function exit as performing the check would in fact be impossible (it may access the value of capture objects that have never been constructed). Consider:

```
void f()
  post [a = get_a(), b = get_b(), c = get_c()] (pred(a, b, c))   // (1)
  post [d = get_d(), e = get_e(), f = get_f()] (pred(d, e, f));  // (2)
```

In the example above, if all contract assertions are evaluated with the `observe` semantic, and `get_b()` exits via an exception, the order of evaluation is as follows:

- The captures at (1) are evaluated – `a` gets constructed, constructing `b` throws;
- The contract-violation handler is called; assuming that it returns normally,
- `a` is destroyed,
- The captures at (2) are evaluated – `d`, `e`, and `f` get constructed,
- the body of `f` is executed,
- the postcondition predicate at (1) is skipped,
- the postcondition predicate at (2) is evaluated,
- `f`, `e`, and `d` are destroyed,
- control is returned to the caller of `f`.

If constructing `b` throws and subsequently the contract-violation handler itself exits via an exception, the postcondition captures that have been constructed are destroyed as part of stack unwinding as normal, in reverse order of construction. If constructing or destroying a postcondition capture throws an exception and the constructor is `noexcept(true)`, `std::terminate` is called as usual. If the function itself exits via an exception and destroying a postcondition capture throws another exception, `std::terminate` is called as usual. If constructing or destroying a postcondition capture exits via an exception and the evaluation semantic is `quick-enforce`, the program is terminated immediately as usual; no further constructors or destructors of postcondition captures are called. If constructing or destroying a postcondition capture results in `longjmp` being called, program termination, the thread being suspended indefinitely, etc., then those effects happen as normal, consistent with the existing C++26 rules for evaluating contract assertions.

5.6.2 Constructing or destroying the capture fails at compile time

If a postcondition capture would copy or destroy an object but it is not copyable or destructible, the program is ill-formed, irrespective of the evaluation semantic chosen by the implementation:

```
void f(std::unique_ptr<int> ptr)
  post [ptr] (ptr);  // error: unique_ptr is not copyable
template <std::movable T>
T select(T x, T y)
 post [x, y] (r: r == x || r == y);  // error on template instantiation
                                     // unless T is also std::copyable
```

Postcondition captures can also be evaluated during constant evaluation, following the usual rules. If such an evaluation encounters an expression that is not a core constant expression (for example, a copy constructor or destructor that is neither `constexpr` nor `consteval`), a compile-time contract

violation occurs. If the evaluation semantic is `observe`, a diagnostic is issued; if the evaluation semantic is `enforce` or `quick-enforce`, the program is ill-formed.

6 Implementation Experience

Postcondition captures have been implemented in branches of both GCC and Clang that are available on Compiler Explorer: https://godbolt.org/z/na5Pxso1K. Both implementations make postcondition captures available with the use of the `-fcontracts-p3098` command-line flag, or the umbrella `-fcontracts-p3850` flag that enables support for all prototypes that are part of the plan proposed in [P3850R1].

This was the most involved feature implemented on these branches — high effort in GCC and medium-to-high in Clang — and essentially all of that complexity flows from a single property: postcondition captures have a two-phase lifetime. A capture is constructed on function entry, interleaved with precondition checks in lexical order, and then destroyed only after the postcondition predicates have been evaluated on return, in reverse order. This separates construction from use across the entire function body, unlike other contract assertions that run to completion at a single point in time, and every implementation mechanism exists to bridge that gap.

Both implementations also had a number of locations where logic previously triggered on having preconditions and now had to also consider postconditions with captures. The GCC implementation has support for outlined checks where all preconditions and postconditions of a function are placed into synthesized helper functions. To support captures, each postcondition assertion with captures needed a synthesized struct that contained the captured variables. These structs are created on the caller's stack when needed, passed to the preconditions function for initialisation, and passed to the postcondition function for use by the predicates and destruction. Each capture is, essentially, stored as an `optional` to track when initialisation was successful so that the right objects would be destroyed if the stack was unwound and postcondition predicates would only be evaluated if the capture initialisation had succeeded.

Minor changes to the contracts ABI were needed to support the new enumeration values, but other than that there is nothing novel about how violations for postconditions with captures are reported.

7 Proposed wording

First, some notes on the design of the wording:

- In order to handle applying the one definition rule to more pieces of a contract assertion we now define the *constituent statements* of a contract assertion that are statements which we can shove into an imaginary function definition and then compare with the odr. This approach makes the statements ones that should, in theory, form a well-defined imaginary function that behaves as we want.
- Determining whether a contract assertion has been violated can now involve multiple evaluations at different points in time. In order to do that we have made a shift in terminology from "evaluation of a contract assertion" to "execution of a contract assertion", where an execution potentially consists of multiple evaluations. For existing behaviors, that is broken into two

evaluations — possibly evaluating the predicate, then possibly executing the contract-violation handling process. Postcondition captures add two more to the list — initialising and destroying the captures.

- This paper includes some of the wording changes from [P3097R3] which has not yet been merged into the draft. Because of the heavy overlap in [expr.call], the wording is included here as well.

These wording changes are relative to the C++29 draft sources with git hash d58cd35f, last modified on Thu, 23 Jul 2026 14:08:45 +0200.

Modified Section Contents

6 Basics **[basic] 25**
6.1 Preamble [basic.pre] 25
6.4 Scope [basic.scope] 26
6.4.2 Point of declaration [basic.scope.pdecl] 26
6.4.10 Contract-assertion scope [basic.scope.contract] 26
6.8 Memory and objects [basic.memobj] 26
6.8.4 Lifetime [basic.life] 26
6.8.6 Storage duration [basic.stc] 27
6.8.6.4 Automatic storage duration [basic.stc.auto] 27
6.10 Program execution [basic.exec] 27
6.10.1 Sequential execution [intro.execution] 28
6.11 Contract assertions [basic.contract] 28
6.11.2 Evaluation [basic.contract.eval] 28

7 Expressions **[expr] 30**
7.5 Primary expressions [expr.prim] 30
7.5.5 Names [expr.prim.id] 30
7.5.5.1 General [expr.prim.id.general] 30
7.5.5.2 Unqualified names [expr.prim.id.unqual] 31
7.5.5.3 Qualified names [expr.prim.id.qual] 32
7.6 Compound expressions [expr.compound] 32
7.6.1 Postfix expressions [expr.post] 32
7.6.1.3 Function call [expr.call] 32

8 Statements **[stmt] 35**
8.8 Jump statements [stmt.jump] 35
8.8.4 The `return` statement [stmt.return] 35
8.9 Assertion statement [stmt.contract.assert] 36

9 Declarations **[dcl] 36**
9.4 Function contract specifiers [dcl.contract] 36
9.4.1 General [dcl.contract.func] 36
9.4.1+a Postcondition Captures [dcl.contract.capture] 38

13 Templates **[temp] 38**
13.7 Template declarations [temp.decls] 38
13.7.4 Variadic templates [temp.variadic] 38

14 Exception handling **[except] 39**
14.3 Stack unwinding [except.ctor] 39

15 Preprocessing directives **[cpp] 40**
15.12 Predefined macro names [cpp.predefined] 40

17 Language support library **[support] 40**
17.10 Contract-violation handling [support.contract] 40
17.10.1 Header `<contracts>` synopsis [contracts.syn] 40
17.10.2 Enumerations [support.contract.enum] 41

Modifications

6 Basics [basic]

6.1 Preamble [basic.pre]

Modify subclause 6.1[basic.pre], paragraph 5:

5 Every name is introduced by a *declaration*, which is a

(5.1) — *name-declaration*, *block-declaration*, or *member-declaration* (9.1[dcl.pre], 11.4[class.mem]),

(5.2) — *init-declarator* (9.3[dcl.decl]),

(5.3) — *identifier* in a structured binding declaration (9.7[dcl.struct.bind]),

(5.3+a) — *identifier* in a *postcondition-capture* in a postcondition assertion (9.4.1+a[dcl.contract.capture]),

(5.4) — *identifier* in a *result-name-introducer* in a postcondition assertion (9.4.2[dcl.contract.res]),

(5.5) — *init-capture* (7.5.6.3[expr.prim.lambda.capture]),

(5.6) — *condition* with a *declarator* (8.1[stmt.pre]),

(5.7) — *member-declarator* (11.4[class.mem]),

(5.8) — *using-declarator* (9.10[namespace.udecl]),

(5.9) — *parameter-declaration* (9.3.4.6[dcl.fct], 13.2[temp.param]),

(5.10) — *type-parameter* (13.2[temp.param]),

(5.11) — *type-tt-parameter* (13.2[temp.param]),

(5.12) — *variable-tt-parameter* (13.2[temp.param]),

(5.13) — *concept-tt-parameter* (13.2[temp.param]),

(5.14) — *elaborated-type-specifier* that introduces a name (9.2.9.5[dcl.type.elab]),

(5.15) — *class-specifier* (11.1[class.pre]),

(5.16) — *enum-specifier* or *enumerator-definition* (9.8.1[dcl.enum]),

(5.17) — *exception-declaration* (14.1[except.pre]), or

(5.18) — implicit declaration of an injected-class-name (11.1[class.pre]).

[*Note 3*: The term declaration is not a synonym for the grammar non-terminal *declaration* (9.1[dcl.pre]). — *end note*]

[*Note 4*: The interpretation of a *for-range-declaration* produces one or more of the above (8.6.5[stmt.ranged]). — *end note*]

6.4 Scope [basic.scope]

6.4.2 Point of declaration [basic.scope.pdecl]

Add a new paragraph after 6.4.2[basic.scope.pdecl], paragraph 12:

12+a The locus of a *postcondition-capture* (9.4.1+a[dcl.contract.capture]) is the start of the predicate of the associated postcondition assertion.

6.4.10 Contract-assertion scope [basic.scope.contract]

Modify section 6.4.10[basic.scope.contract], paragraph 1:

1 Each contract assertion (6.11[basic.contract]) C introduces a *contract-assertion scope* that includes

(1.a) — the predicate of C, and

(1.b) — the initializer of any postcondition-captures of C.

Note

The above bulletted list is broken down to avoid conflicts with P3400, which will introduce an expression as part of a contract assertion that is explicitly not part of the contract-assertion scope.

6.8 Memory and objects [basic.memobj]

6.8.4 Lifetime [basic.life]

Modify section 6.8.4[basic.life], paragraph 11:

11 If a program ends the lifetime of an object of type `T` with static (6.8.6.2[basic.stc.static]), thread (6.8.6.3[basic.stc.thread]), or automatic (6.8.6.4[basic.stc.auto]) storage duration and if `T` has a non-trivial destructor,[17] and another object of the original type does not occupy that same storage location when the implicit destructor call takes place, the

behavior of the program is undefined (F.2.12[ubx:original.type.implicit.destructor]). This is true even if the ~~block is exited with an exception~~destruction occurs as part of stack unwinding.

[*Example 4*:

```
class T { };
struct B {
  ~B();
};

void h() {
  B b;
  new (&b) T;
}                                   // undefined behavior at block exit
```

— *end example*]

---

[17] That is, an object for which a destructor will be called implicitly—upon exit from the block, completion of the full expression, or completion of the postcondition assertion execution for an object with automatic storage duration, upon exit from the thread for an object with thread storage duration, or upon exit from the program for an object with static storage duration.

6.8.6 Storage duration [basic.stc]

6.8.6.4 Automatic storage duration [basic.stc.auto]

Modify section 6.8.6.4[basic.stc.auto], paragraphs 1-2:

1 Variables that belong to a block scope and are not explicitly declared `static`, `thread_local`, or `extern` have *automatic storage duration*. The storage for such variables lasts until the block in which they are created exits.

[*Note 1*: These variables are initialized and destroyed as described in 8.10[stmt.dcl]. — *end note*]

Variables that belong to a parameter scope also have automatic storage duration. The storage for a function parameter lasts until immediately after its destruction (7.6.1.3[expr.call]). Postcondition captures also have automatic storage duration. The storage for a postcondition capture lasts until immediately after the execution of the associated postcondition completes (???[basic.contract.capture], 6.11.2[basic.contract.eval]).

2 If a variable with automatic storage duration has initialization or a destructor with side effects, an implementation shall not destroy it ~~before the end of its block~~except when specified nor eliminate it as an optimization, even if it appears to be unused, except that a class object or its copy/move may be eliminated as specified in 11.9.6[class.copy.elision].

Note

Ths reads as "we will do the things we say we will do. It is possible this paragraph should be made a note instead.

6.10 Program execution [basic.exec]

6.10.1 Sequential execution [intro.execution]

Modify section 6.10.1[intro.execution], paragraph 11:

11 When invoking a function f (whether or not the function is inline), every argument expression and the postfix expression designating f are sequenced before every precondition ~~assertion~~evaluation of the function call (7.6.1.3[expr.call]), which in turn are sequenced before every expression or statement in the body of f, which in turn are sequenced before every postcondition ~~assertion~~evaluation of the function call.

6.11 Contract assertions [basic.contract]

6.11.2 Evaluation [basic.contract.eval]

Add paragraphs to and modify section 6.11.2[basic.contract.eval], paragraphs 4-7:

3+a The *execution of a contract assertion* A consists of a (possibly empty) sequence of evaluations that are sequenced in order. The evaluations are determined by the evaluation semantic as described below.

3+b All of the evaluations that are part of an execution of a contract assertion form a block that is suspended between the evaluations.

[*Note 1+a*: Each execution, therefore, has its own instance of each object declared by any postcondition captures associated with the contract assertion. — *end note*]

4 The ~~evaluation~~execution of ~~a contract assertion~~ A using the ignore semantic consists of a single evaluation that has no effect.

[*Note 2*: The predicate is potentially evaluated (6.3[basic.def.odr]), but not evaluated. — *end note*]

4+a The execution of A using a checking semantic consists of

(4+a.1) — the initialization of each postcondition capture of A, in order,

(4+a.2) — the evaluation of the predicate of A,

(4+a.3) — the occurrence of a contract violation based on the result of A, and

(4+a.4) — the destruction of each postcondition capture of A, in reverse order.

5 The ~~evaluation A~~execution of ~~a contract assertion~~ A using a checking semantic determines the value of the predicate. It is unspecified whether the ~~predicate is evaluated~~evaluations of the contract assertion other than a potential contract violation are performed. Let B be the value of the predicate that would result from performing ~~evaluating~~the evaluations of the ~~predicate~~contract assertion.

[*Note 3*: To determine whether a predicate would evaluate to `true` or `false`, ~~an~~ alternative ~~evaluation~~evaluations that ~~produces~~produce the same ~~value~~result as the predicate but ~~has~~have no side effects can occur.

[*Example 1*:

```
struct S {
  mutable int g = 5;
} s;
```

```
void f()
  pre(( s.g++, false ));    // #1
void g()
{
  f();  // Increment of s.g might not occur, even if #1 uses a checking semantic.
}
```

— *end example*]

— *end note*]

6 There is an observable checkpoint (4.1.2[intro.abstract]) *CO* that happens before each of the evaluations of *A* such that any other evaluation that happens before ~~*A*~~that evaluation also happens before *CO*.

7 A *contract violation* of *A* occurs when

(7.1) — *B* is `false`,

(7.2) — ~~the~~any evaluation of ~~the predicate~~*A* exits via an exception, or

(7.3) — ~~the~~any evaluation of ~~the predicate~~*A* is performed in a context that is manifestly constant-evaluated (7.7.7[expr.const.defns]) and ~~the predicate~~that evaluation is not a core constant expression.

[*Note 4*: If *B* is `true`, no contract violation occurs and control flow continues normally ~~after the point of evaluation of the contract assertion~~. The evaluation of the predicate can fail to produce a value without causing a contract violation, for example, by calling `longjmp` (17.14.3[csetjmp.syn]) or terminating the program. Destruction of postcondition captures can result in a contract violation even after *B* was determined to be `true`. — *end note*]

Modify section 6.11.2[basic.contract.eval], paragraph 12:

12 If the contract violation occurred because ~~the~~an evaluation ~~of the predicate~~ exited via an exception, the contract-violation handler is invoked from within an active implicit handler for that exception (14.4[except.handle]). If the contract-violation handler returns normally and the evaluation semantic is observe, that implicit handler is no longer considered active. No further evaluations that are part of the current execution of that contract assertion other than the destruction of postcondition captures that have already been initialized are performed.

[*Note 9*: The exception can be inspected or rethrown within the contract-violation handler. — *end note*]

Modify section 6.11.2[basic.contract.eval], paragraphs 17-18:

17 If a contract-violation handler invoked from ~~the~~an evaluation of a function contract assertion (9.4.1[dcl.contract.func]) exits via an exception, the behavior is as if the function body exits via that same exception.

[*Note 12*: A *function-try-block* (14.1[except.pre]) is the function body when present and thus does not have an opportunity to catch the exception. If the function has a non-throwing exception specification, the function `std::terminate` is invoked (14.6.2[except.terminate]). — *end note*]

[*Note 13*: If a contract-violation handler invoked from an *assertion-statement* (8.9[stmt.contract.assert]) exits via an exception, the search for a handler continues from the execution of that statement. — *end note*]

18 ~~To *evaluate in sequence*~~The *evaluation sequence* of a list R of contract assertions~~:~~ is a list S such that

(18.a) — all elements of R are in S,

(18.1) — ~~Construct a list of contract assertions S such that~~

~~— all elements of R are in S,~~

— each element of R may be repeated an implementation-defined number of times within S, and

~~— if a contract assertion A precedes another contract assertion B in R, then the first occurrence of A precedes the first occurrence of B in S.~~

(18.2) — ~~Evaluate each element of S such that,~~ if a contract assertion A precedes ~~a~~another contract assertion B in SR, then the first ~~evaluation~~occurrence of A ~~is sequenced~~precedes ~~before~~ the ~~evaluation~~first occurrence of B in S.

[*Example 3*:

```
void f(int i)
{
  contract_assert(i > 0);   // #1
  contract_assert(i < 10);  // #2
    // valid sequence of evaluations: #1 #2
    // valid sequence of evaluations: #1 #1 #2 #2
    // valid sequence of evaluations: #1 #2 #1 #2
    // valid sequence of evaluations: #1 #2 #2 #1
    // invalid sequence of evaluations: #2 #1
}
```

— *end example*]

7 Expressions [expr]

7.5 Primary expressions [expr.prim]

7.5.5 Names [expr.prim.id]

7.5.5.1 General [expr.prim.id.general]

Modify section 7.5.5.1[expr.prim.id.general], paragraph 2:

2 If an *id-expression* E denotes a non-static non-type member of some class `C` at a point where the current class (7.5.3[expr.prim.this]) is `X` and

(2.1) — E is potentially evaluated or `C` is `X` or a base class of `X`, and

(2.2) — E is not the *id-expression* of a class member access expression (7.6.1.5[expr.ref]), and

(2.3) — E is not the *id-expression* of a *reflect-expression* (7.6.2.10[expr.reflect]), and

(2.4) — if E is a *qualified-id*, E is not the un-parenthesized operand of the unary `&` operator (7.6.2.2[expr.unary.op]),

the *id-expression* is transformed into a class member access expression using `(*this)` as the object expression. If this transformation occurs in

(2.4+a) — the predicate of a precondition assertion, or

(2.4+b) — the initializer of a postcondition capture

of a constructor of `X`, or the predicate of a postcondition assertion of a destructor of `X`, the expression is ill-formed.

[*Note 2*: If `C` is not `X` or a base class of `X`, the class member access expression is ill-formed. Also, if the *id-expression* occurs within a static or explicit object member function, the class member access is ill-formed. — *end note*]

This transformation does not apply in the template definition context (13.8.3.2[temp.dep.type]).

[*Example 1*:

```
struct C {
  bool b;
  C() pre(b)                // error
      pre(&this->b)         // OK
      pre(sizeof(b) > 0);   // OK, b is not potentially evaluated.
};
```

— *end example*]

7.5.5.2 Unqualified names [expr.prim.id.unqual]

Modify section 7.5.5.2[expr.prim.id.unqual], paragraph 7:

7 Otherwise, if the *unqualified-id* appears in

(7.a) — the predicate, or

(7.b) — the initializer of a postcondition capture

of a contract assertion C (6.11[basic.contract]) and the entity is

(7.1) — a variable declared outside of C of object type `T`,

(7.2) — a variable or template parameter declared outside of C of type "reference to `T`", or

(7.3) — a structured binding of type `T` whose corresponding variable is declared outside of C,

then the type of the expression is `const T`.

Note

If we choose to apply `const`-ification to postcondition captures we would add that as a separate bullet here.

Modify section 7.5.5.2[expr.prim.id.unqual], paragraphs 13-14:

13 If an *id-expression* E appears in the predicate of a function contract assertion attached to a function f and denotes a function parameter of f and the implementation introduces any temporary objects to hold the value of that parameter as specified in 6.8.7[class.temporary],

(13.1) — if the ~~contract assertion~~evaluation is a precondition ~~assertion and the~~ evaluation ~~of the precondition assertion~~that is sequenced before the initialization of the parameter object, E refers to the most recently initialized such temporary object, and

(13.2) — if the ~~contract assertion~~evaluation is a postcondition ~~assertion~~evaluation, it is unspecified whether E refers to one of the temporary objects or the parameter object; the choice is consistent within a ~~single~~complete evaluation of a postcondition assertion.

14 If an *id-expression* E names a result binding ~~in a postcondition assertion~~R and the implementation introduces any temporary objects to hold the result object as specified in 6.8.7[class.temporary], ~~and the postcondition assertion~~then for any evaluation of E that is sequenced before the initialization of the result object denoted by R (7.6.1.3[expr.call]), E refers to the most recently initialized such temporary object.

7.5.5.3 Qualified names [expr.prim.id.qual]

Modify section 7.5.5.3[expr.prim.id.qual], paragraph 7:

7 If Q appears in

(7.a) — the predicate, or

(7.b) — the initializer of a postcondition capture

of a contract assertion C (6.11[basic.contract]) and the entity is

(7.1) — a variable declared outside of C of object type `T`,

(7.2) — a variable declared outside of C of type "reference to `T`", or

(7.3) — a structured binding of type `T` whose corresponding variable is declared outside of C,

then the type of the expression is `const T`.

Note

If we choose to apply `const`-ification to postcondition captures we would add that as a separate bullet here.

7.6 Compound expressions [expr.compound]

7.6.1 Postfix expressions [expr.post]

7.6.1.3 Function call [expr.call]

Add a new paragraph after 7.6.1.3[expr.call], paragraph 4:

4+a The precondition assertions of a function call are, in order:

(4+a.1) — if the *postfix-expression* is a (possibly implicit) class member access expression and the call is a virtual function call, the precondition assertions (9.4.1[dcl.contract.func]) of the statically chosen function;

(4+a.2) — the precondition assertions of the function actually called.

The postcondition assertions of a function call are, in order:

(4+a.3) — the postcondition assertions of the function actually called;

(4+a.4) — if the *postfix-expression* is a (possibly implicit) class member access expression and the call is a virtual function call, the postcondition assertions of the statically chosen function.

[*Example a*:

```
struct X1 { virtual void f()          pre(a) post(b) {} };
struct X2 { virtual void f()          pre(c) post(d) {} };
struct Y : X1    { void f() override pre(e) post(f) {} };
struct Z : Y, X2 { void f() override pre(g) post(h) {} };

void t() {
  Z z;
  z.f();                          // asserts g, h
  static_cast<Y*>(&z)->f();       // asserts e, g, h, f

  X1& x1ref = z;
  X2& x2ref = z;
  x1ref.f();                      // asserts a, g, h, b
  x2ref.f();                      // asserts c, g, h, d
  x1ref.X1::f();                  // asserts a, b
  x1ref.[:^^X1::f:]();            // asserts a, g, h, b

  void (X1::*pmf)() = &X1::f;
  (x1ref.*pmf)();                 // asserts g, h
}
```

— *end example*]

4+b The *precondition evaluations* and *postcondition evaluations* of a function call consist of the evaluations of an execution of a contract assertion C for every C in an evaluation sequence X of the precondition assertions and an evaluation sequence Y of the postcondition assertions of the function call. Each such evaluation is in the precondition evaluations of the function call if

(4+b.1) — it is the evaluation of the predicate of a precondition assertion,

(4+b.2) — it is the occurrence of a contract violation based on the result of a precondition assertion, or

(4+b.3) — it is the initialization of a postcondition capture,

otherwise it is in the postcondition evaluations.

[*Note 3+a*: An evaluation is in the postcondition evaluations if

(4+b.4) — it is the evaluation of the predicate of a postcondition assertion,

(4+b.5) — it is the occurrence of a contract violation based on the result of a postcondition assertion, or

(4+b.6) — it is the destruction of a postcondition capture.

— *end note*]

4+c The order of the precondition evaluations and the postcondition evaluations of a function call have the following constraints:

(4+c.1) — The evaluations associated with a single execution of a contract assertion are in the same order that they are specified (6.11.2[basic.contract.eval]).

(4+c.2) — The evaluation of a predicate of a contract assertion and the occurrence of a contract violation of a contract assertion are in the same order as the corresponding contract assertions in X or Y.

(4+c.3) — The evaluation of the predicate of a precondition assertion A shall be after the first evaluation of the initialization of any postcondition capture associated with a postcondition assertion B for any B in Y that precedes A in lexical order.

(4+c.4) — The initialization of a postcondition capture associated with a postcondition assertion B shall be after the first evaluation associated with a precondition assertion A for any A in X that precedes B in lexical order.

[*Note 3+b*: The evaluation of the destruction of postcondition captures happens in the reverse order of their initialization, but it can be interleaved with the evaluations of the predicates of other postcondition assertions. — *end note*]

Modify section 7.6.1.3[expr.call], paragraphs 7-10:

7 When a function is called, each parameter (9.3.4.6[dcl.fct]) is initialized (9.5[dcl.init], 11.4.5.3[class.copy.ctor]) with its corresponding argument, and each precondition ~~assertion~~ evaluation of the function call ~~is evaluated~~occurs (9.4.1[dcl.contract.func]). If the function is an explicit object member function and there is an implied object argument (12.2.2.2.2[over.call.func]), the list of provided arguments is preceded by the implied object argument for the purposes of this correspondence. If there is no corresponding argument, the default argument for the parameter is used.

[*Example 2*:

```
template<typename ...T> int f(int n = 0, T ...t);
int x = f<int>();               // error: no argument for second function parameter
```

— *end example*]

If the function is an implicit object member function, the object expression of the class member access shall be a glvalue and the implicit object parameter of the function (12.2.2[over.match.funcs]) is initialized with that glvalue, converted as if by an explicit type conversion (7.6.3[expr.cast]).

[*Note 5*: There is no access or ambiguity checking on this conversion; the access checking and disambiguation are done as part of the (possibly implicit) class member access operator. See 6.5.2[class.member.lookup], 11.8.3[class.access.base], and 7.6.1.5[expr.ref]. — *end note*]

When a function is called, the type of any parameter shall not be a class type that is either incomplete or abstract.

[*Note 6*: This still allows a parameter to be a pointer or reference to such a type. However, it prevents a passed-by-value parameter to have an incomplete or abstract class type. — *end note*]

It is implementation-defined whether a parameter is destroyed when the function in which it is defined exits (8.8.4[stmt.return], 14.3[except.ctor], 7.6.2.4[expr.await]) or at the end of the enclosing full-expression; parameters are always destroyed in the reverse order of their construction. The initialization and destruction of each parameter occurs within the context of the full-expression (6.10.1[intro.execution]) where the function call appears.

[*Example 3*: The access (11.8.1[class.access.general]) of the constructor, conversion functions, or destructor is checked at the point of call. If a constructor or destructor for a function parameter throws an exception, any *function-try-block* (14.1[except.pre]) of the called function with a handler that can handle the exception is not considered. — *end example*]

8 The *postfix-expression* is sequenced before each *initializer-clause* in the *expression-list* and any default argument. The initialization of a parameter or, if the implementation introduces any temporary

objects to hold the values of function parameters (6.8.7[class.temporary]), the initialization of those temporaries, including every associated value computation and side effect, is indeterminately sequenced with respect to that of any other parameter. These evaluations are sequenced before the ~~evaluation of the~~ precondition ~~assertions~~evaluations of the function ~~call, which are evaluated in sequence (9.4.1[dcl.contract.func])~~. For any temporaries introduced to hold the values of function parameters, the initialization of the parameter objects from those temporaries is indeterminately sequenced with respect to the evaluation of each precondition assertion.

[*Note 7*: All side effects of argument evaluations are sequenced before the function is entered (see 6.10.1[intro.execution]). — *end note*]

[*Example 4*:

```
void f() {
  std::string s = "but I have heard it works even if you don't believe in it";
  s.replace(0, 4, "").replace(s.find("even"), 4, "only").replace(s.find(" don't"), 6, "");
  assert(s == "I have heard it works only if you believe in it");       // OK
}
```

— *end example*]

[*Note 8*: If an operator function is invoked using operator notation, argument evaluation is sequenced as specified for the built-in operator; see 12.2.2.3[over.match.oper]. — *end note*]

[*Example 5*:

```
struct S {
  S(int);
};
int operator<<(S, int);
int i, j;
int x = S(i=1) << (i=2);
int y = operator<<(S(j=1), j=2);
```

After performing the initializations, the value of `i` is 2 (see 7.6.7[expr.shift]), but it is unspecified whether the value of `j` is 1 or 2. — *end example*]

9 The result of a function call is the result of the possibly-converted operand of the `return` statement (8.8.4[stmt.return]) that transferred control out of the called function (if any), except in a virtual function call if the return type of the final overrider is different from the return type of the statically chosen function, the value returned from the final overrider is converted to the return type of the statically chosen function.

10 When the called function exits normally (8.8.4[stmt.return], 7.6.2.4[expr.await]), all postcondition ~~assertions~~ evaluations of the function call ~~are evaluated in sequence~~occur (9.4.1[dcl.contract.func]). If the implementation introduces any temporary objects to hold the result value as specified in 6.8.7[class.temporary], ~~the evaluation of~~ each postcondition ~~assertion~~evaluation is indeterminately sequenced with respect to the initialization of any of those temporaries or the result object. These evaluations, in turn, are sequenced before the destruction of any function parameters.

8 **Statements** [stmt]

8.8 **Jump statements** [stmt.jump]

8.8.4 **The `return` statement** [stmt.return]

Modify section 8.8.4[stmt.return], paragraph 5:

5 The copy-initialization of the result of the call is sequenced before the destruction of temporaries at the end of the full-expression established by the operand of the `return` statement, which, in turn, is sequenced before the destruction of local variables (8.8[stmt.jump]) of the block enclosing the `return` statement.

[*Note 3*: These operations are sequenced before the destruction of local variables in each remaining enclosing block of the function (8.10[stmt.dcl]), which, in turn, is sequenced before the ~~evaluation of~~ postcondition ~~assertions~~ evaluations of the function call (9.4.1[dcl.contract.func]), which, in turn, is sequenced before the destruction of function parameters (7.6.1.3[expr.call]). — *end note*]

8.9 Assertion statement [stmt.contract.assert]

Modify subclause 8.9[stmt.contract.assert], paragraph 3:

3 The evaluation of consecutive *assertion-statement*s ~~is~~executes all evaluations of each contract assertion in an evaluation ~~in~~ sequence (6.11.2[basic.contract.eval]) ofthe contract assertions introduced by those *assertion-statement*s.

[*Note 1*: A sequence of *assertion-statement*s can thus be repeatedly evaluated as a group.

[*Example 1*:

```
int f(int i)
{
  contract_assert(i == 0);  // #1
  contract_assert(i >= 0);  // #2
  return 0;
}
int g = f(0);   // can evaluate #1, #2, #1, #2
```

— *end example*]

— *end note*]

9 Declarations [dcl]

9.4 Function contract specifiers [dcl.contract]

9.4.1 General [dcl.contract.func]

Modify section 9.4.1[dcl.contract.func]:

function-contract-specifier-seq:
function-contract-specifier function-contract-specifier-seq$_{opt}$

function-contract-specifier:
precondition-specifier
postcondition-specifier

precondition-specifier:
`pre` *attribute-specifier-seq*$_{opt}$ (*conditional-expression*)

postcondition-specifier:
`post` *attribute-specifier-seq*$_{opt}$ *postcondition-captures-clause*$_{opt}$ (*result-name-introducer*$_{opt}$ *conditional-expression*)

Add a paragraph to and modify section 9.4.1[dcl.contract.func], paragraph 5:

4+a The *constituent statements* of a contract assertion are, in order:

(4+a.1) — For every postcondition capture associated with the contract assertion, if any, a variable declaration whose name, type, and initializer are the same as the postcondition capture.

(4+a.2) — If there is a *result-name-introducer*, a variable declaration whose name and type are the same as the result binding.

(4+a.3) — An `if` statement whose condition is the predicate of the contract assertion.

5 A *function-contract-specifier-seq* SF_1 is the same as a *function-contract-specifier-seq* SF_2 if SF_1 and SF_2 consist of the same *function-contract-specifier*s in the same order. A *function-contract-specifier* C_1 on a function declaration D_1 is the same as a *function-contract-specifier* C_2 on a function declaration D_2 if

(5.1) — their ~~predicates~~constituent statements PS_1 and PS_2 would satisfy the one-definition rule (6.3[basic.def.odr]) if placed in function definitions on the declarations D_1 and D_2, respectively, except for

(5.1.1) — renaming of the parameters of f,

(5.1.2) — renaming of template parameters of a template enclosing f, and

(5.1.3) — renaming of the result binding (9.4.2[dcl.contract.res]), if any,

and, if D_1 and D_2 are in different translation units, corresponding entities defined within each predicate behave as if there is a single entity with a single definition, and

(5.2) — both C_1 and C_2 specify a *result-name-introducer* or neither do.

If this condition is not met solely due to the comparison of two *lambda-expression*s that are contained within PS_1 and PS_2, no diagnostic is required.

[*Note 3*: Equivalent *function-contract-specifier-seq*s apply to all uses and definitions of a function across all translation units. — *end note*]

[*Example 1*:

```
bool b1, b2;

void f() pre (b1) pre ([]{ return b2; }());
void f();                       // OK, function-contract-specifiers omitted
void f() pre (b1) pre ([]{ return b2; }()); // ill-formed, no diagnostic required: closures have
                                            // different types
void f() pre (b1);              // error: function-contract-specifiers only partially repeated

int g() post(r : b1);
int g() post(b1);       // error: mismatched result-name-introducer presence

namespace N {
  void h() pre (b1);
  bool b1;
  void h() pre (b1);    // error: function-contract-specifiers differ according to
                        // the one-definition rule (6.3[basic.def.odr]).
}
```

— *end example*]

9.4.1 + *a* **Postcondition Captures** **[dcl.contract.capture]**

Add a new section 9.4.1+a[dcl.contract.capture] after 9.4.1[dcl.contract.func]

9.4.1+a Postcondition Captures [dcl.contract.capture]

postcondition-captures-clause:
    [*postcondition-capture-list*]
postcondition-capture-list:
    postcondition-capture
    postcondition-capture-list , *postcondition-capture*
postcondition-capture:
    postcondition-parameter-capture
    postcondition-init-capture
postcondition-parameter-capture:
    identifier ...$_{opt}$
postcondition-init-capture:
    ...$_{opt}$ *identifier initializer*

1 A *postcondition capture* is associated with a postcondition assertion and declares a variable that inhabits the contract-assertion scope (6.4.10[basic.scope.contract]) of its associated postcondition assertion.

A *postcondition-capture* containing an ellipsis is a pack expansion that declares a postcondition-capture pack (13.7.4[temp.variadic]).

The *identifier* of a *postcondition-parameter-capture* shall denote a parameter of the function that the postcondition assertion is associated with. If the *postcondition-parameter-capture* contains an ellipsis the parameter shall be a parameter pack. The initializer of the introduced postcondition capture is the identifier.

[*Note 1*: The variable introduced by a *postcondition-parameter-capture* inhabits the contract-assertion scope of the postcondition assertion and thus the *identifier* in the initializer will refer to the function parameter and not the capture. — *end note*]

Each postcondition capture introduces a variable with automatic storage duration whose placeholder type is `auto` and whose type is deduced from its initializer.

[*Note 2*: In the initializer, an expression naming a variable, including a function parameter, will have `const` added to its type (7.5.5.3[expr.prim.id.qual], 7.5.5.2[expr.prim.id.unqual]). — *end note*]

Note

Here is a place we will need to modify when we make a final decision about whether the type of the variable should be const and whether it should be 'const'-ified in the predicate.

13 **Templates** **[temp]**

13.7 **Template declarations** **[temp.decls]**

13.7.4 **Variadic templates** **[temp.variadic]**

Add a new paragraph after 13.7.4[temp.variadic], paragraph 3:

3+a A *postcondition-capture pack* is a *postcondition-capture* that introduces a *postcondition-capture* for each of the elements in the pack expansion of its *initializer*.

[*Note a*: The initializer of a *postcondition-parameter-capture* is its *identifier* and will denote a function parameter pack. — *end note*]

[*Example 3+a*:

```
template <typename T> T transform(const T& t);
template <typename... Ts> bool check(Ts... ts);

template <typename... Args>
void foo(Args... args)
  post [ args... ] (check(args...))     // args in predicate refer to captures of each function parameter
  post [ ...targs=transform(args) ]
                   (check(targs...));   // targs is a pack of one capture for each element of args.
```

— *end example*]

14 Exception handling [except]

14.3 Stack unwinding [except.ctor]

Notes regarding 14.3[except.ctor], paragraph 2:

2 Each object with automatic storage duration is destroyed if it has been constructed, but not yet destroyed, since the try block was entered. If an exception is thrown during the destruction of temporaries or local variables for a `return` statement (8.8.4[stmt.return]), the destructor for the returned object (if any) is also invoked. The objects are destroyed in the reverse order of the completion of their construction.

[*Example*:

```
struct Y { ~Y() noexcept(false) { throw 0; } };

A f() {
  try {
    A a;
    Y y;
    A b;
    return {};      // #1
  } catch (...) {
  }
  return {};        // #2
}
```

At #1, the returned object of type `A` is constructed. Then, the local variable `b` is destroyed (8.8[stmt.jump]). Next, the local variable `y` is destroyed, causing stack unwinding, resulting in the destruction of the returned object, followed by the destruction of the local variable `a`. Finally, the returned object is constructed again at #2. — *end example*]

Note

The above code handles destroying objects during stack unwinding, but it is not immediately obvious which actual automatic objects are destroyed by this process. (All of them everywhere? Probably not.) We want to be sure to include those created by postcondition captures, but it is not clear exactly how to accomplish that.

15 **Preprocessing directives** [cpp]

15.12 **Predefined macro names** [cpp.predefined]

Modify subclause 15.12[cpp.predefined], Table 22 [cpp.predefined.ft]:

Table 1: Table 22 — Feature-test macros [tab:cpp.predefined.ft]

Macro name	Value
⋮ *20 rows elided* ⋮	
`__cpp_constinit`	201907L
`__cpp_contracts`	~~202606L~~202502L
`__cpp_contracts_postcondition_captures`	xxxxxxL
`__cpp_decltype`	200707L
`__cpp_decltype_auto`	201304L
⋮ *56 rows elided* ⋮	

17 **Language support library** [support]

17.10 **Contract-violation handling** [support.contract]

17.10.1 **Header `<contracts>` synopsis** [contracts.syn]

Modify section 17.10.1[contracts.syn], paragraph 1:

1 The header `<contracts>` defines types for reporting information about contract violations (6.11.2[basic.contract.eval]).

```
// all freestanding
namespace std::contracts {

  enum class assertion_kind : unspecified {
    pre = 1,
    post = 2,
    assert = 3,
    post_capture = 4↩
  };

  enum class evaluation_semantic : unspecified {
    ignore = 1,
    observe = 2,
    enforce = 3,
    quick_enforce = 4
  };

  enum class detection_mode : unspecified {
    predicate_false = 1,
    evaluation_exception = 2
  };

  class contract_violation {
    // no user-accessible constructor
  public:
```

```
    contract_violation(const contract_violation&) = delete;
    contract_violation& operator=(const contract_violation&) = delete;

    see below ~contract_violation();

    const char* comment() const noexcept;
    contracts::detection_mode detection_mode() const noexcept;
    bool is_terminating() const noexcept;
    assertion_kind kind() const noexcept;
    source_location location() const noexcept;
    evaluation_semantic semantic() const noexcept;
  };

  void invoke_default_contract_violation_handler(const contract_violation&);
}
```

17.10.2 **Enumerations** [**support.contract.enum**]

Modify section 17.10.2[support.contract.enum], Table 44 [support.contract.enum.kind]:

Table 44 — Enum `assertion_kind` [**tab:support.contract.enum.kind**]

Name	Meaning
`pre`	A precondition assertion
`post`	~~A~~Evaluating the predicate of a postcondition assertion
`assert`	An *assertion-statement*
`post_capture`	Construction or destruction of a postcondition capture

Bibliography

[N1613] Thorsten Ottosen, “Proposal to add Design by Contract to C++”, 2004
http://wg21.link/N1613

[N1669] Thorsten Ottosen, “Proposal to add Contract Programming to C++ (revision 1)”, 2004
http://wg21.link/N1669

[N1773] D. Abrahams, L. Crowl, T. Ottosen, and J. Widman, “Proposal to add Contract Programming to C++ (revision 2)”, 2005
http://wg21.link/N1773

[N1866] L. Crowl and T. Ottosen, “Proposal to add Contract Programming to C++ (revision 3)”, 2005
http://wg21.link/N1866

[N4110] J. Daniel Garcia, “Exploring the design space of contract specifications for C++”, 2014
http://wg21.link/N4110

[P0465R0] Lisa Lippincott, “Procedural function interfaces”, 2016
http://wg21.link/P0465R0

[P0542R5] J. Daniel Garcia, “Support for contract based programming in C++”, 2018
http://wg21.link/P0542R5

[P2461R1] Gašper Ažman, Caleb Sunstrum, and Bronek Kozicki, “Closure-based Syntax for Contracts”, 2021
http://wg21.link/P2461R1

[P2885R3] Timur Doumler, Joshua Berne, Gašper Ažman, Andrzej Krzemieński, Ville Voutilainen, and Tom Honermann, “Requirements for a Contracts syntax”, 2023
http://wg21.link/P2885R3

[P2900R11] Joshua Berne, Timur Doumler, and Andrzej Krzemieński, “Contracts for C++”, 2024
http://wg21.link/P2900R11

[P2900R14] Joshua Berne, Timur Doumler, and Andrzej Krzemieński, “Contracts for C++”, 2025
http://wg21.link/P2900R14

[P2957R2] Andrzej Krzemieński, Iain Sandoe, Joshua Berne, and Timur Doumler, “Contracts and coroutines”, 2024
http://wg21.link/P2957R2

[P2961R2] Timur Doumler and Jens Maurer, “A natural syntax for Contracts”, 2023
http://wg21.link/P2961R2

[P3088R1] Timur Doumler and Joshua Berne, “Attributes for contract assertions”, 2024
http://wg21.link/P3088R1

[P3097R2] Timur Doumler and Joshua Berne, “Contracts for C++: Virtual functions”, 2026
http://wg21.link/P3097R2

[P3097R3] Timur Doumler, Joshua Berne, and Gašper Ažman, “Contracts for C++: Virtual functions”, 2026
http://wg21.link/P3097R3

[P3387R0] Timur Doumler, Joshua Berne, Iain Sandoe, and Peter Bindels, “Contract assertions on coroutines”, 2024
http://wg21.link/P3387R0

[P3484R2] Timur Doumler and Joshua Berne, “Postconditions odr-using a parameter modified in an overriding function”, 2024
http://wg21.link/P3484R2

[P3487R0] Timur Doumler and Joshua Berne, “Postconditions odr-using a parameter that may be passed in registers”, 2024
http://wg21.link/P3487R0

[P3510R1] Nathan Myers and Gašper Ažman, “Leftover properties of `this` in constructor preconditions”, 2024
http://wg21.link/P3510R1

[P3850R0] Timur Doumler and Joshua Berne, "A proposed plan for extending Contracts in C++29", 2026
http://wg21.link/P3850R0

[P3850R1] Timur Doumler and Joshua Berne, "A proposed plan for extending Contracts in C++29", 2026
http://wg21.link/P3850R1